

Dynamic programming (2D)

Recap

1. **Define subproblems.** $dp[i]$ = "best" way to handle $nums[0..i]$
2. **Find recurrence.** $dp[i] = f(dp[i-1], dp[i-2], \dots)$
3. **Specify order.** $i = 0..n-1$ or $i = n-1..0$?
4. **Base case.** $dp[0] = ?$

Min path sum

$c=0$ $c=1$ $c=2$

$r=0$

1	3	1
---	---	---

$r=1$

1	5	1
---	---	---

$r=2$

4	2	1
---	---	---

$$dp(1,1) =$$

lowest cost from
for left to
(1,1)

$$= 1 + 1 + 5$$

$$= 7$$

$$dp(2,1) = 8$$

Min path sum

1	3	1
1	5	1
4	2	1

Step 1. Define subproblems

$dp[r][c]$ = Min cost to get from top left to $[r,c]$

Step 1. Define subproblems

1	3	1
1	5	1
4	2	1

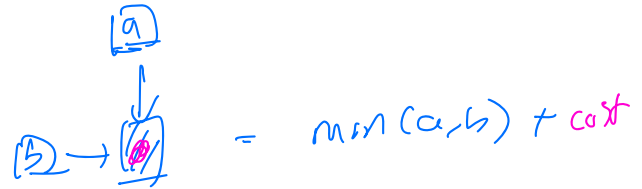
input
cost

tc

1	4	5
2	7	6
6	8	7

Path 2
dp

path 1
↓
cheapest
+



Step 2. Find recurrence

$$\text{dp}[r][c] = \text{cost}[r][c] + \min(\text{dp}[r-1][c], \text{dp}[r][c-1])$$

best way to get to
top

↓
left

Step 2. Find recurrence

1	3	1
1	5	1
4	<u>2</u>	1

1	4	5
2	7	6
6	8	7

$$\begin{aligned}
 & \text{input cost} \\
 & dp(r, c) = \text{cost}(r, c) + \min(dp(\text{left}), dp(\text{top})) \\
 & \quad \quad \quad \downarrow 2 \\
 & \quad \quad \quad = 2 + \min(6, 7) = 8
 \end{aligned}$$

Step 3. Iteration order

$$dp[r][c]$$

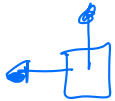
$$dp[r-1, c]$$

$$dp[r, c-1]$$

for $r = 0 \rightarrow n-1$

for $c = 0 \rightarrow n-1$

$dp[r, c] = \dots$



1	4	5
2	7	6
6	8	7

dp

Step 4. Base case

$$dp[r][c] = cost[r][c] + \min(dp[r-1][c], dp[r][c-1])$$

need
base
cases for...

r or $c = 0$

Code (spot the bug!)

```
dp = [[float('inf')] * C for _ in range(R)]
dp[0][0] = grid[0][0] → min cost to get to top left

for r in range(R):
    for c in range(C):

        from_up = dp[r-1][c] if r-1 >= 0 else 0
        from_left = dp[r][c-1] if c-1 >= 0 else 0
        dp[r][c] = grid[r][c] + min(from_up, from_left)

return dp[-1][-1]
```

↑ (NE
bug?)

Dangerous attempt: set to 0

$$dp[r][c] = cost[r][c] + \min(dp[r-1][c], dp[r][c-1])$$

1	3	1
1	5	1
4	2	1

input

1	4	5
2	7	6
6	8	7

dp

~~0~~ + 3 = 3

1 + 3 = 4

Safer: Omit invalid options

```
dp[r][c] = WORST_VALUE
for each option:
    if option is invalid, skip
    else dp[r][c] = min(dp[r][c], valid options' value)
```

Code

```
dp = [[float('inf')] * C for _ in range(R)]
dp[0][0] = grid[0][0]

for r in range(R):
    for c in range(C):
        if r-1 >= 0:
            dp[r][c] = min(dp[r][c], grid[r][c] + dp[r-1][c])
        if c-1 >= 0:
            dp[r][c] = min(dp[r][c], grid[r][c] + dp[r][c-1])

return dp[-1][-1] → ? min cost from top left to bottom right
```

Runtime: $O(R \cdot C)$
Space complexity: $O(R \cdot C)$
optimized space complexity: ?
 $O(C)$

optimized space complexity: need previous row/col.

$O(R \cdot C)$.

$\text{cost} = 0$
for $i = 1 \rightarrow n$
for $j = 1 \rightarrow n$
cost $++$

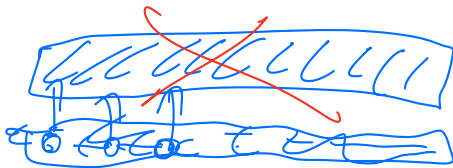
$=$

if $n = 10$
cost = 100

cost = 0
for each day of the week:
for each of 24 hours
cost $++$

$=$ cost = 7×24

$r = 1$



$dp(r)(?) \dots$

$O(\min(R, C))$

Demo time

Tracing actions

```
option_taken[2][2] = (1, 2)
```

1	4	5
2	7	6
6	8	7

dp

opt_taken(0,2) = (0,1)

opt_taken(1,2) = (0,2)

Code without tracing

```
dp = [[float('inf')] * C for _ in range(R)]
dp[0][0] = grid[0][0]

for r in range(R):
    for c in range(C):
        if r-1 >= 0:
            dp[r][c] = min(dp[r][c], grid[r][c] + dp[r-1][c])
        if c-1 >= 0:
            dp[r][c] = min(dp[r][c], grid[r][c] + dp[r][c-1])

return dp[-1][-1]
```

option taken: (1, 2, 3)

if $exp1 < dp[r][c]$:
 $dp[r][c] = \min(dp[r][c], dp[r-1][c])$

Tracing actions (in general)

```
option_taken[r][c]
dp[r][c] = WORST_VALUE
for each option:
    if option is invalid, skip
    else
        if valid options' value < dp[r][c]:
            dp[r][c] = valid option's value
            option_taken[r][c] = option
```

Tracing actions (for this problem)

```
option_taken = [[None] * C for _ in range(R)]  
... Same base cases  
  
for r in range(R):  
    for c in range(C):  
        if r-1 >= 0:  
            option_val = grid[r][c] + dp[r-1][c]  
            if option_val < dp[r][c]:  
                dp[r][c] = option_val  
                option_taken[r][c] = (r-1,c)  
        if c-1 >= 0:  
            ... Similar code block
```

Reconstructing the path

```
option_taken[2][2] = (1, 2)
```

1	4	5
2	7	6
6	8	7

opt_taken (2,2)

dp

print_path (takes opt_taken())()

① start at opt_taken (n-1, n-1)

↪ start at bottom right

path = [] ② current = (n-1, n-1), while (current != (0,0):
path.add(current)

③ return path.reverse()

current = opt_taken(current)

goal:

(0,0)

↪ (0,1) → (0,2)

↓
(1,2) ↪ (2,2)

dp(r)(c) = ... dp(r-1)(c), dp(r)(c-1)



reconstruct:

① opt_taken ② linked list iteration

Sequence alignment

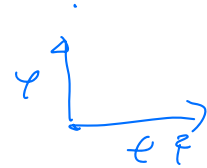
- Applied math. [Dynamic time warping](#)
- Comp bio. [Comparing DNA sequences](#)
- AI. [Viterbi algorithm](#). Audio waves -> most likely words.
- Text similarity. [Edit distance](#)

DP 2D

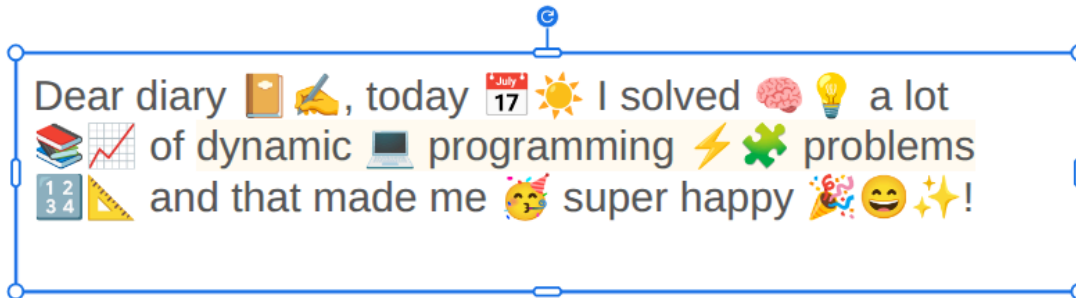
sequence alignment

dynamic time warping
edit distance - text

LCS (longest common subsequence)



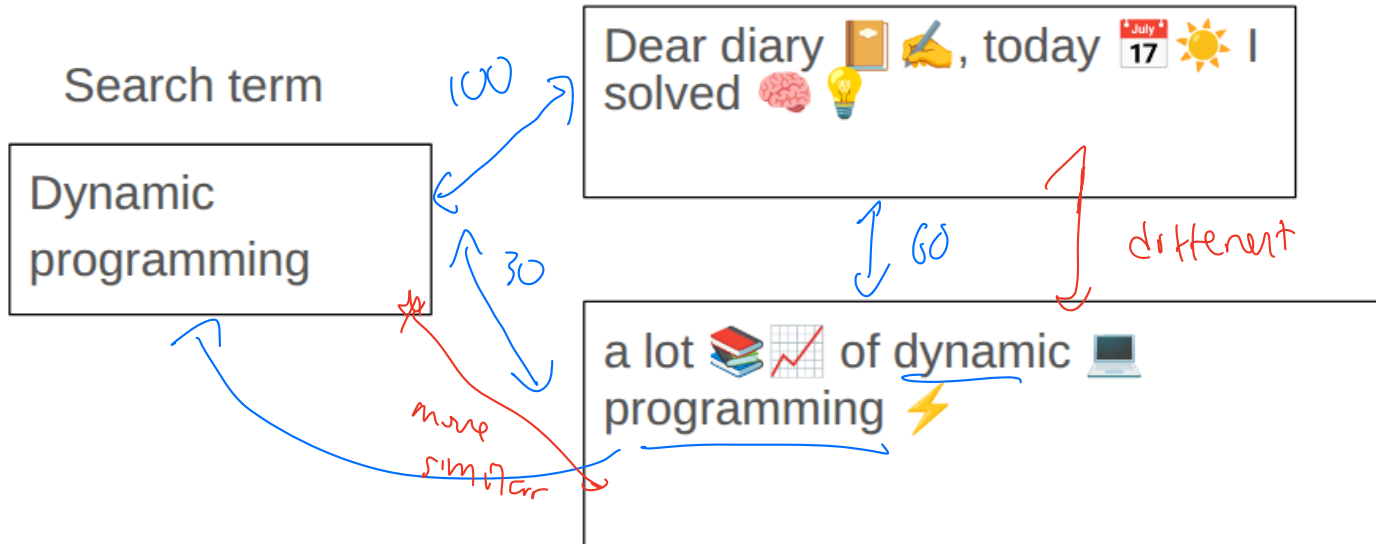
Edit distance



Stephen Jonany
2:56 PM Today

Replace "dynamic programming
with "DP"

Edit distance



Edit distance

Input: Two strings s_1, s_2 .

Output: Min cost of transforming s_1 to s_2 .

Transform: delete, add, substitute.

See UW slide 12

hello $\rightarrow$ hell / 1 deletion
hell $\rightarrow$ hello / 1 addition
hello $\rightarrow$ helly / 1 substitution

Longest common subsequence

subsequence:

vs
substring

s1 = CARPET

def: min deletion on both
to make them equal

s2 = CHART

→ LCS = CART

Leetcode link

s = carpet

arp →

c pet →

substrings?	subsequence
✓	✓
X	✓

Longest common subsequence

Input: Two strings s_1 , s_2 , of length m and n .

Output: The length L of the longest string that is a subsequence of **both**.

z mm #1 deletion, make $s_1^* = s_2^*$

Step 1. Define subproblems

$dp[i][j]$ = length of the LCS of the **first** i characters of $s1$
and the **first** j characters of $s2$

Step 1. Define subproblems

Handwritten notes on the left:

$dp[2][3]$
= LCS of
"CA" and
"CHA"
= 2

Handwritten notes on the right:

$dp[3][4] =$
LCS of
"CAR", "CHAR"
= 3

$dp[0][5] =$
LCS of ""
and
"CHART"
= 0

		0	1	2	3	4	5
	""	C	H	A	R	T	
0	""	0	0	0	0	0	0
1	C	0	1	1	1	1	1
2	A	0	1	1	2	2	2
3	R	0	1	1	2	3	3
4	P	0	1	1	2	3	3
5	E	0	1	1	2	3	3
6	T	0	1	1	2	3	4

CARE
CHAR



Step 2. Find recurrence

Look at the last character of each prefix: $s1[i-1]$ and $s2[j-1]$.

Either they match, or they don't.

Case 1: The last characters match

Claim. If they match, it is never bad to keep them both.

Proof by contradiction. Two subcases:

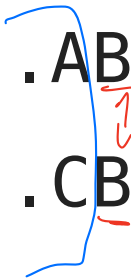
- (a) You delete both *x not good*
- (b) You delete exactly one

C A R P E T
C H A R T

False!

Case 1a: Deleting both is not good

s1 = . . AB
s2 = . . CB



Contradiction:
whatever soln you
come up w/
I can append
- - B
- - B

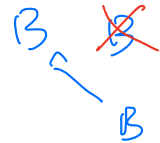
Case 1b: Deleting one is not good

Your soln:

s1 = . . AB

s2 = . . CB

enemy:



No, my soln is always never worse off.

Case 1: The last characters match

---B
 .--B

Proved: It's never bad to keep both characters if they match.

So the value for this option is: $1 + dp[i-1][j-1]$

$\text{cnt } dp[i][j] =$

length of LCS between

$s1[0..i]$

and

$s2[0..j]$

$dp[i-1][j-1] + 1$

$s1 = \dots AB$

$s2 = \dots CB$

if the last chars match

Case 2: The last characters don't match

CATR ~~PET~~
CAT ~~P~~

Then you're forced to delete...

$dp[i][j] = \max \{$

- the top $= dp[i-1][j]$
- bottom $= dp[i][j-1]$
- or both $= dp[i-1][j-1]$

$s1 = \dots AC$

optional!

$s2 = \dots CB$

because

$dp[i][j-1] > dp[i-1][j-1]$

$\max(dp[i-1][j], dp[i][j-1])$

Step 2. Find recurrence

```
if s1[i-1] == s2[j-1]:  
    dp[i][j] = 1 + dp[i-1][j-1]  
else:  
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

(2 ... j)

{
2
3
.
i



dp[i][j] needs:

- ① ← $c_{i,j-1}$
- ② ↑ $c_{i-1,j}$
- ③ ↖ $c_{i-1,j-1}$

Step 3. Iteration order

Base case:

0's:

LCS of empty
strng

w/
any strng
lcs = 0

	""	C	H	A	R	T
""	0	0	0	0	0	0
C	0	1	1	1	1	1
A	0	1	1	2	2	2
R	0	1	1	2	3	3
P	0	1	1	2	3	3
E	0	1	1	2	3	3
T	0	1	1	2	3	4



Step 4. Base cases

- First row: $dp[0][j] = 0$
- First column: $dp[i][0] = 0$

LCS Code

```
dp = [[0] * (n + 1) for _ in range(m + 1)]
```

```
# Base cases: row 0 and column 0 are already 0
```

```
for i in range(1, m + 1):  
    for j in range(1, n + 1):
```

```
        if s1[i-1] == s2[j-1]:  
            dp[i][j] = 1 + dp[i-1][j-1]
```

```
        else:
```

```
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

```
return dp[m][n]
```



$dp(i-1, j-1) + 1$

3
2
1

) if last 2 chars match

) if last 2 chars
don't match
must delete of recur
one

Sequence alignment

ex1. audio alignment.

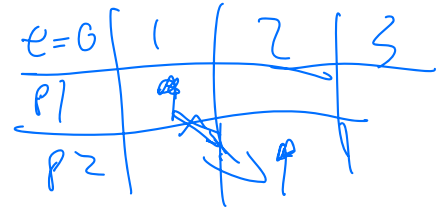
→ sync up everyone's
happy birthday

ex2, syncing video game on

ex3, G1: P1s, P3r, P4dop P3)...
G2: P1s, r-

vrzhr $t=1, D\#$
ether $t=2, D\#$
mannu $t=2, D\#$

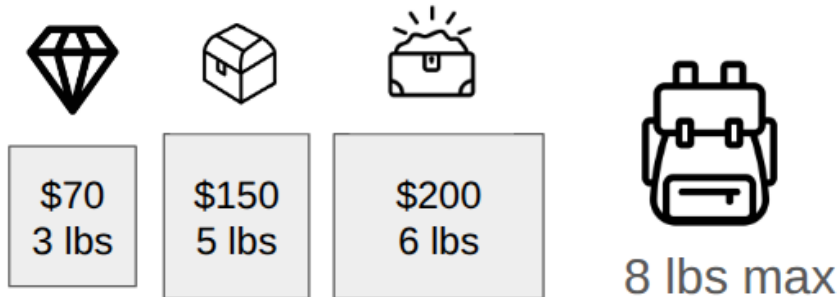
$D\#, P, P, D\#$
 $P\#, P, D\#$



S1:

S2:

Knapsack



Knapsack

Input:

- Item values $v_0 \dots v_{n-1}$, positive numbers
- Item weights $w_0 \dots w_{n-1}$, all positive integers
- Knapsack capacity C

Output:

- Max total collection value, where the total weight is $\leq C$

Step 1. Define subproblems

$dp[i][c]$ = Max total value if we consider items 0 to i ,
and our knapsack capacity is c

Items = (70, 3lb), (150, 5lb), (\$200, 6lb)

Step 1. Define subproblems

$dp[i][c]$ = Max total value if we consider items 0 to i ,
and our knapsack capacity is c

Items = (70, 3lb), (150, 5lb), (\$200, 6lb)

capacity	0	1	2	3	4	5	6	7	8
c									

$i=0$	0	0	0	70	70	70	70	70	70
-------	---	---	---	----	----	----	----	----	----

$i=1$	0	0	0	70	70	150	150	150	220
-------	---	---	---	----	----	-----	-----	-----	-----

$i=2$	0	0	0	70	70	150	200	200	220
-------	---	---	---	----	----	-----	-----	-----	-----

Step 2. Find recurrence

Either the i -item is included, or not.

Step 3. Iteration order

capacity	0	1	2	3	4	5	6	7	8
----------	---	---	---	---	---	---	---	---	---

c

i=0	0	0	0	70	70	70	70	70	70
i=1	0	0	0	70	70	150	150	150	220
i=2	0	0	0	70	70	150	200	200	220

Step 4. Base cases

- First row: $dp[0][*] =$
- First column: $dp[*][0] =$

Code

```
dp = [[0] * (C + 1) for _ in range(N)]

for c in range(C + 1):
    if weights[0] <= c:
        dp[0][c] = values[0]

for i in range(1, N):
    for c in range(C + 1):
        dp[i][c] = dp[i-1][c]
        if c - weights[i] >= 0:
            dp[i][c] = max(dp[i][c], \
                           dp[i-1][c - weights[i]] + values[i])

return dp[N-1][C]
```

Runtime & Space complexity

N = number of items

C = max capacity

Runtime: $O(NC)$

Space complexity: $O(NC)$

Recap

- $dp[r][c]$ = min cost to travel from top left to location $[r][c]$
LCS
- $dp[i][j]$ = min cost to transform $s1[0..i]$ into $s2[0..j]$
Z sequence
- $dp[i][c]$ = max total value if we consider items 0 to i ,
and capacity is c
knapsack